

Mentoria de IA aplicada · **Aula 1 de 8**

Como LLMs funcionam

Tokens, sampling, context window e alucinação

Hoje

Ao final, você consegue:

- explicar **por que** o mesmo prompt dá respostas diferentes
- estimar **tokens e custo** de uma chamada
- dizer o que acontece quando a conversa **não cabe** no contexto
- explicar **por que** o modelo alucina

Roteiro (40 min)

1. Gancho
2. Tokens
3. Um token por vez + sampling
4. Contexto, custo e latência
5. Alucinação
6. Trailer da leitura de apoio

Depois: 15 min de discussão

Qual destas é a resposta certa?

demo:temperatura

Escreva a primeira frase de uma newsletter semanal sobre IA. — mesmo prompt, 5 execuções

1. Olá! Aqui está o destaque da semana sobre inteligência artificial.
2. O que a inteligência artificial está mudando nas suas rotinas na semana?
3. O que a inteligência artificial está inventando na semana que vem?
4. O que a inteligência artificial está inventando na semana?
5. A semana na era da inteligência artificial começou com grandes avanços...

qwen3:4b-instruct, temperature 1.2. Rode ao vivo: o resultado muda a cada execução.

Um LLM é uma função que recebe **tokens** e devolve uma **distribuição de probabilidade** sobre o próximo token.

Todo o resto vem de **sortear dessa distribuição, um token por vez.**

1 • Tokens

O que o modelo realmente enxerga

O modelo não lê palavras

demo:tokens

0 modelo prevê o próximo token com base no contexto .

11 tokens

Parcelamento sem juros no cartão de crédito

8 tokens · em inglês, "Interest-free credit card installments" = 5

inconstitucionalmente

1 palavra, 5 tokens

Por que isso importa

Custo

Você paga **por token**, não por palavra.

Limite

A janela de contexto é medida em **tokens**.

Idioma

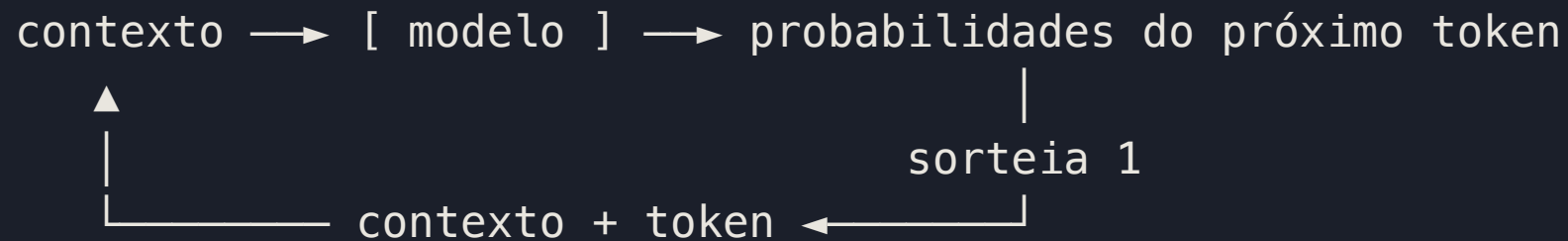
O mesmo texto pode virar **até 15x** mais tokens em alguns idiomas.

Contar letras ou inverter uma palavra é difícil para quem enxerga [Parcelamento] .

2 · Um token por vez

Geração autoregressiva e sampling

Geração autoregressiva



- O texto aparece aos poucos (**streaming**) porque é gerado aos poucos
- Não há "plano": cada token olha só o que **já foi escrito**
- Uma escolha ruim no começo **puxa** as seguintes

O próximo token, ao vivo demo:proximo-token

Complete a frase com poucas palavras: A capital do Brasil é

Token escolhido	Alternativas (probabilidade)
" é"	" é" 100%
" Bras"	" Bras" 50,4% · " **" 49,3% · " Rio" 0,3%
"ília"	"ília" 100%

A dúvida do modelo não estava no fato: estava em abrir ou não um negrito em markdown.

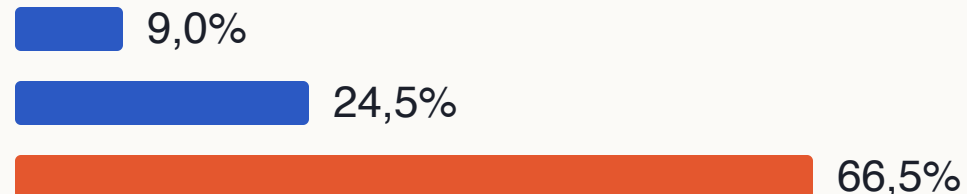
Temperatura: o quanto confiar no favorito

Logits [1, 2, 3] → softmax

T = 0,5 (concentra)



T = 1



T → 0 = sempre o favorito (**greedy**)

T alto = distribuição achatada,
azarões ganham chance

$$p_i = \exp(z_i / T) / \sum \exp(z_j / T)$$

Você implementa isso no exercício.

Top-p: corta a cauda

- Mantém o **menor grupo** de tokens mais prováveis que soma pelo menos p
- Sorteia **só entre eles**
- Remove os tokens improváveis, onde mora o texto sem sentido

Escolher sempre o mais provável gera texto "sem graça e estranhamente repetitivo"; sortear da distribuição inteira gera lixo. Top-p fica no meio.

Voltando ao gancho demo:temperatura

temperature 0

5 de 5 iguais:

"Olá, seja bem-vindo à sua newsletter semanal sobre Inteligência Artificial!"

temperature 1.2

5 de 5 diferentes.

Nenhuma é "a certa": são **sorteios** da mesma distribuição.

Classificar, extrair, gerar código → temperatura baixa · **Brainstorm** → mais alta

⚠ temperature 0 reduz a variação, mas não garante saída idêntica em todo provedor.

3 · Contexto, custo e latência

Context window

- O modelo **não tem memória** entre chamadas: tudo que ele "sabe" da conversa está no prompt
- Num chat, o **histórico inteiro** é reenviado a cada mensagem
- Quando não cabe, **alguém corta**: o provedor recusa, a biblioteca corta ou você corta
- Caber não é o mesmo que ser bem usado: o desempenho **cai** quando a informação relevante está no **meio** de contextos longos

Custo e latência

Custo

```
custo = entrada × preço_entrada  
       + saída   × preço_saída  
       (por milhão de tokens)
```

Exemplo com preços **hipotéticos** (US\$ 2 / US\$ 8 por milhão):
1.500 tokens de entrada + 300 de saída
= **US\$ 0,0054**

Latência

- **Entrada:** processada em paralelo → afeta o tempo até o 1º token
- **Saída:** um token por vez → resposta longa = resposta lenta

Para alto volume: peça respostas **curtas e estruturadas**

4 • Alucinação

Plausível não é verdadeiro

Por que o modelo inventa

- O modelo gera o **texto mais plausível** dado o contexto
- Se a pergunta cita um artigo que **não existe**, a continuação mais plausível de "resuma o artigo X" é... **um resumo**
- A resposta falsa sai com o **mesmo tom confiante** da verdadeira

"O modelo mentiu" é impreciso. Ele **completou** o texto, e não havia nada no mecanismo que o obrigasse a checar fatos.

Três perguntas sobre coisas que não existem demo:alucinacao

Artigo inventado

Resumiu "Gradientes Tropicais" (NeurIPS 2019) com detalhes técnicos.

Invenção total

Framework inventado

Negou que "Jabuticaba.js" orquestre agentes... e descreveu um framework web.

Invenção parcial

Cidade inventada

Disse que "Porto Esmeralda do Norte" não existe... e citou um complexo turístico em Pernambuco.

Negou + inventou detalhe

Por que ela persiste

- **No pré-treino:** se o modelo não distingue afirmações incorretas de fatos, erros surgem por pressão estatística
- **Nas avaliações:** a maioria dos benchmarks **premia chutar** em vez de admitir incerteza, como prova sem desconto por erro

O que fazer (o resto da trilha)

Fornecer a fonte no contexto · validar a saída (**M2–M3**) · usar a incerteza para mandar casos duvidosos a um humano (**M7**)

Na leitura de apoio

Embeddings

Tokens viram **vetores**; significados parecidos ficam perto.

Atenção

Cada token **olha os outros** do contexto para se entender.

Pré-treino → RLHF → RLVR

Como um "completador de texto" vira um modelo que **segue instruções**.

Tudo em `conceitos.md` e na **apostila** do módulo.

Discussão

15 minutos

Para conversar

1. Se `temperature 0` dá sempre a mesma resposta, por que não usar 0 sempre?
2. O projeto final vai classificar notícias. Que temperatura você usaria?
3. Um chatbot "esqueceu" o começo da conversa. Quais são as explicações possíveis?
4. "O modelo mentiu." Qual seria uma descrição mais precisa?
5. Se a saída custa mais e sai um token por vez, o que muda no seu prompt para uma tarefa de alto volume?

Para a próxima aula

Exercício

softmax , top-p, amostrar , corte de contexto

```
pnpm --filter @mentoria/ex01-llms test
```

Desafio: mini modelo de bigramas que **alucina**

Projeto final · M1

Classificar notícias em **texto livre**

```
pnpm --filter @mentoria/curador m01:temperaturas
```

Traga o arquivo `saidas/m01-temperaturas.md`

Próxima aula · Além do chat: System One e Jev

E se, em vez de texto, o modelo devolvesse uma **decisão tipada com confiança?**

Na M1, com temperature 1.5, o formato da resposta já começa a variar: "Relevantes →", "JUSTIFICATIVA:". Um programa que lê isso com regex quebra.